ChatGPT

Deep research on China LLM post-training and agentic RL interview questions and interview-experience posts

Summary conclusions

Across the most "round-realistic" public posts (i.e., entries that include a specific company, a concrete round like "1st interview / 2nd interview," and project drill-down), China's 2025–2026 LLM algorithm interviews are converging on one dominant axis: **post-training alignment algorithms** (PPO / DPO / GRPO and increasingly DAPO / GSPO) plus **hands-on debugging, stability, and reward design** rather than only textbook definitions. This is visible in multiple recent interview-experience posts that explicitly ask for "PPO vs GRPO vs DPO," "GRPO/GSPO changes and formulas," "advantage/baseline," "reward collapse," and "what's wrong if GRPO loss becomes 0," with follow-ups that force you to reason about the training loop and failure modes—not just recite. [1]

The second major axis is that **infra depth is now treated as a differentiator for post-training candidates**: interviews frequently push into **verl config semantics (rollout.n, global batch vs micro batch, PPO epochs / mini-batch splitting), DeepSpeed ZeRO stage tradeoffs, MoE train–serve mismatch, and inference engines like vLLM (KV cache, PagedAttention, continuous batching)**. In other words, "have you used verl?" has become "explain the interplay between rollout and update batch sizes; explain why remove-padding matters; compare ZeRO-1/2/3; compute KV cache size; explain PagedAttention." [2]

For agentic roles, interviewers are less satisfied with "Agent = ReAct + tools." They increasingly probe **tool-chain scheduling, fallback behavior, memory update/forgetting, planning vs hallucination metrics, and evaluation gates**. A notable 2025–2026 shift is that **MCP (Model Context Protocol)** shows up directly in candidate experiences and "what is MCP / how is it different from function calling / skills" question sets, reflecting a broader ecosystem move toward standardizing tool integrations. [3]

Finally, **project interrogation is often the true selection mechanism**: interviewers repeatedly dig into "what did you actually ship," "what was the DAU/cost/latency," "how did you A/B test," "how did you construct and validate SFT/RL data," "how did you generate 'hard negatives'," and "how did you verify improvements weren't reward hacking." Several real posts show explicit questions on A/B testing, resource/count-of-GPUs used for RL, data volumes for SFT vs RL, and evaluation logic for choosing better candidates (responses) from multiple generations. [4]

Site value ranking

In practice, different Chinese platforms reliably map to different "signal types" (real interview rounds vs curated question banks vs learning templates). The ranking below follows your priority order, but also reflects what was actually discoverable and time-stamped in 2025–2026.

Highest value for "real rounds / real drill-down": nowcoder.com (牛客).
The core reason is not volume; it's that many posts include **exact timestamps** and dense round-like

question lists with **project follow-ups** (e.g., explicit questions about verl config parameters, RL batch sizes, “how many nodes,” A/B details, CoT length handling). Examples span multiple companies and 2025–2026 dates. [5]

Second tier for “explanations + curated interview templates”: juejin.cn and CSDN.
These sites are strong for **structured “how to answer” writeups** (Agent loops, tool calling logic, RLHF overviews, algorithm comparison explainers), but these are typically **not round-accurate** and often read like “answer keys” rather than an interviewer’s actual prompts. They are best used to fill gaps after you’ve extracted authentic question patterns from 牛客. [6]

Third tier for “infra truth sources”: GitHub + official docs / arXiv.
For infra-heavy interviews, the highest-confidence preparation materials are the **actual repos and docs** (verl docs for GRPO config semantics; vLLM repo and the PagedAttention paper; ZeRO paper for reasoning about DeepSpeed stages). These sources don’t tell you what a specific interviewer asked last week, but they directly support the “whiteboard config + memory math” questions that repeatedly appear in real interview posts. [7]

Supplemental tier: zhihu.com and bilibili.com.
Zhihu often contains long-form explainers (useful if you cross-check against primary papers), while Bilibili provides fast “pattern recognition” content (e.g., GRPO/verl parameter explanations). However, both have a higher ratio of **repackaging / content marketing**; treat them as supporting material rather than primary “what is actually asked” evidence. [8]

Xiaohongshu: only via reprints that are searchable and publicly accessible.
A good example is a 牛客 post labeled as a Xiaohongshu multimodal interview, which is publicly indexed and contains concrete post-training and data/RLAIF prompts. [9]

Timeliness judgment

This section uses **March 13, 2026** as “today,” and buckets by “time since posting date” as displayed on the pages.

Within the last 90 days (Dec 13, 2025 – Mar 13, 2026): the center of gravity is GRPO-variants + verl configs + agent tool standards.
You see explicit questions about GRPO/DAPO/GSPO comparisons and improvements, GSPO-vs-GRPO formula and training stability, and even “if GRPO loss becomes 0, is it normal and how to debug.” You also see fast-rising agent questions around MCP tooling, tool-response loss masking, and “Agentic training average output tokens and compression.” [10]
Newer infra prompts in this window are strongly config- and performance-oriented (remove-padding, MoE train–serve mismatch, DeepSpeed stage specifics, vLLM/serving memory concepts). [11]

Within 90–180 days (Sep 14, 2025 – Dec 13, 2025): “PPO/DPO/GRPO as baseline literacy,” plus a stronger push into “read the original paper” and failure modes.
Multiple posts in late 2025 still revolve around the PPO/DPO/GRPO family, but the interviewer style often shifts to “what did you learn from the original paper,” “how does DPO relate to reward model training,” and “how do you reason about KV cache and attention variants for efficiency.” [12]

Within 180–365 days (Mar 14, 2025 – Sep 13, 2025): reasoning-centric post-training begins to surface, but interviews are relatively less config-heavy than 2026.
Posts in mid-2025 already ask about PPO vs GRPO and “why everyone is focusing on reasoning,” and

begin referencing DeepSeek-R1-style reasoning improvements; but compared to early 2026, there are fewer “verl knob deep dives” and fewer MCP/tool-chain standard questions visible in the public interview-experience corpus. [13]

Older than 365 days (before Mar 14, 2025): lower weight for your target.
Older materials are still useful for fundamentals, but their surface forms often differ from 2026’s “systems + stability + evaluation” emphasis. A practical example is a 2025-01 post that looks more like a generic question list (parameter counting, evaluation methods) and less like the 2026 config/debug grill. [14]

Inferred “what’s rising” in the last ~6 months (high confidence from multiple recent posts): - **GRPO variants (DAPO/GSPO/DSPO) and their motivations,** plus CoT length / length-bias handling. [15]
- **verl as a de-facto RL training stack**, with questions that demand config-level understanding. [16]
- **MCP / standardized tool integration** appearing as named interview topics for agent roles. [17]

Stable “never goes away” topics (high confidence): - PPO vs DPO comparisons, KL control, reward/critic reasoning, and preference data construction. [18]
- KV cache, serving latency/cost reasoning, and paging-style memory management for inference (vLLM/PagedAttention). [19]
- LoRA/PEFT basics remain a cross-role staple, repeatedly cited as high frequency even in curated “count tables.” [20]

Topics that look lower-weight or “declining” (based on curated frequency summaries, not purely real rounds): - Older PEFT variants like Adapter/Prefix-Tuning are explicitly described as low-frequency/ “declining” in one Alibaba-focused summary compilation. Treat this as weaker evidence than multi-company real posts, but consistent with broader community discourse. [21]

High-frequency theme panorama

The table below follows your requested 10-topic taxonomy, assigns **S/A/B** priority for post-training + agentic RL job-seeking, and notes whether the topic is “fast-changing” (watch last 3–6 months) or “stable.”

Theme cluster (your taxonomy)	Priority for post-training + agentic RL	Why this priority (interview behavior)	Time sensitivity
Post-training / RL fundamentals	A	Frequently used as the “warm-up,” but interviewers quickly escalate into algorithm-specific math and system constraints. [22]	Medium (concepts stable; examples shift toward reasoning models) [23]
RL algorithm details (PPO/DPO/GRPO/DAPO/GSPO…)	S	The single most repeated family: interviewers ask to compare, justify selection, and explain improvements/edge cases. [24]	High (variants and “best practice” conventions change fast) [25]

Theme cluster (your taxonomy)	Priority for post-training + agentic RL	Why this priority (interview behavior)	Time sensitivity
Reward design / advantage / critic / KL / entropy / reward hacking	S	Real rounds explicitly probe reward function design, critic vs reward model, advantage/baseline, collapse/hacking patterns, and debugging. [26]	High (new failure-mode narratives appear with new algos and reasoning tasks) [25]
verl / vLLM / DeepSpeed / Ray / distributed training & training systems	S	Infra questions are no longer optional: several interviews ask for verl training flow and explicit config parameter relationships; plus ZeRO stages and efficiency tricks (remove-padding). [27]	Medium–High (specific stacks and knobs evolve) [28]
Agent / tool use / planning / memory / evaluation / safety	S	Real agent interviews drill scheduling/fallback, memory update, evaluation metrics, and tooling standards (MCP) and even training-time loss masking for tool traces. [29]	High (tool protocols and evaluation norms change quickly) [30]
RAG / retrieval / rerank / long context	A	Appears repeatedly in agent interviews and is often asked as "how do you evaluate RAG, how do you handle duplicates, rerank choice." [31]	Medium (tech evolves, but core design questions persist) [32]
Transformer / MoE / attention / RoPE / RMSNorm, etc.	A	Still heavily tested, often with "write it or compute it" expectations; also tied to efficiency (GQA/MQA, RMSNorm, FlashAttention, MoE). [33]	Medium (architectural trends shift slowly; specific model reports change faster) [34]
Inference & serving / latency / cost / KV cache / PagedAttention	A	Strongly present in infra lists; also appears in algorithm interviews as soon as you mention production. PagedAttention and KV cache questions are common. [35]	Medium–High (serving stacks move quickly; principles stable) [36]
Project deep dive (why this algo / metrics / improvements)	S	Interviewers repeatedly "drill until it hurts": AB, DAU/cost, data volumes, evaluation gates, and resource configs. [37]	Medium (questions stable; what counts as "good evidence" trends upward) [38]
Live coding / DS&A	A	Still present across roles (MHA hand-implement, prefix sums, classic LeetCode). Even infra interviews often add DS&A. [39]	Low (problem types stable)

Company-organized leads

Below are “leads” (high-signal threads) organized by company, with a **realness label** per item:

- **Type 1: Real interview experience** (specific company + date + round-like list + project grill)
- **Type 2: Question-bank / compilation** (aggregated, frequency tables, “530+ questions,” etc.)
- **Type 3: Technical summary / answer template** (teaches concepts, not necessarily a real round)

Tencent
Type 1: A recent internship interview list pushes data filtering for alignment, GRPO/DAPO/GSPO comparison, routing/difficulty estimation (“router vs classifier”), CoT length control, and multi-stage RL. [40]
Type 1: A campus “fail experience” explicitly probes agentic training token length/compression and GSPO-vs-GRPO (plus DSPO awareness). [41]
Type 1: A WXG internship list drills AB testing, traffic/cost, SFT vs RL data volume, RL cluster resources, and “GSPO vs GRPO + what if GRPO loss is zero.” [42]
Type 1 (older): A Hunyuan-oriented post includes RLHF workflow questions (why RLHF after SFT), and “DeepSeek-style GRPO vs PPO” comparisons. [43]

Baidu
Type 1: The “Wenxin Yiyan” interview explicitly asks PPO vs GRPO pros/cons, DPO training curve behavior, DeepSpeed and ZeRO communication, and reasoning-focused questions (“why reasoning now; how to improve base model reasoning”). [44]
Type 1: A 2026 “Wenxin base-model AI infra” interview lead is very infra-heavy: “what changes did you make in verl source,” DeepSpeed ZeRO1–3, remove-padding, MoE train–serve mismatch, and tensor/sequence parallelism. [45]

Alibaba group (Taobao / “TaoTian”) + Amap (AutoNavi)
Type 1: A Taobao Agent interview lead combines GRPO/GSPO training flow questions with RAG system modules, rerank-vs-embedding choice, memory update/forgetting, and production traffic/cost optimization. [46]
Type 1 (app-agent): A Taobao Agent campus interview asks modular-agent planning, tool-chain scheduling and fallback, and a clean split between planning ability and hallucination metrics. [47]
Type 1: Amap internship questions explicitly mention GRPO KL control, padding-loss handling in SFT, and “logprobs in consistency reward.” [48]
Type 2: A 2026 Alibaba-focused compilation claims frequency counts (e.g., RoPE/RMSNorm/KV cache/PagedAttention; DPO/RLHF/Reward Modeling; ZeRO stages), which is useful for triangulation but should not be treated as “ground truth across all China companies.” [49]

Ant Group
Type 1: A high-signal post drills verl training flow and explicit config parameter relationships (rollout/global/micro batch sizes and rollout.n), plus reward function design and critic necessity; also asks for “recent RL methods” including GRPO/DAPO/GSPO improvements. [50]
This is one of the strongest matches to your “verl infra + post-training” preference because it explicitly tests the **knob-to-effective-sample-count reasoning** and the **reward/critic story**. [51]

ByteDance ecosystem
Type 1: A ByteDance internship interview asks DPO/PPO/GRPO, reward collapse, offline vs online RL, GRPO math details, and multi-objective reward conflicts; it also includes MHA hand-implementation. [52]
Type 1: Another ByteDance post (multimodal) asks PPO/GRPO/GSPO/DAPO evolution, train–serve

mismatch for GSPO, plus “what is ReAct” and agent-based data generation. [53]
Type 3 (but important for infra credibility): the verl repo states it is initiated by ByteDance Seed and supports building GRPO/PPO flows while integrating with FSDP/Megatron/vLLM/SGLang. [54]

Meituan
Type 1: The “LLM RL first interview” list spans RMSNorm, FlashAttention, model-report understanding, Mamba, and explicitly asks whether you know verl. [55]
Type 1: A second-round post emphasizes deep project params (GPU config, training params) and requires comparing GRPO/DPO/DAPO/GSPO. [56]

Kuaishou
Type 1: A daily internship post asks KV cache mechanics, decoder QKV details, MHA variants (GQA/MQA/MLA), and explicitly asks for DPO/PPO/GRPO/DAPO formulas plus “did you read original papers.” [57]
Type 1: Another Kuaishou post touches multimodal post-training and multimodal RAG. [58]
Type 1 (search snippet only): a Kuaishou 2nd-interview snippet includes reward hacking and RM training difficulty—highly relevant, but the full page wasn’t accessible in-text during this run. [59]

MiniMax
Type 1: A recent “2nd interview” lead focuses on report-generation pipelines, schema constraints, retrieval alignment, synthetic data construction, ranking logic (rule vs model), reward model usage, and evaluation dimensions. This strongly matches “tool-use + evaluation + data synthesis.” [60]
Type 1 (older): a 2025-01 intern post is more generic (parameter counting, evaluation methods, decoding), thus lower-weight for your “post-training + infra” target but still useful as a baseline. [14]

Zhipu
Type 1: An “agent algorithm” interview is exceptionally aligned with your preferences: DPO loss intuition, why not SFT, negative-sample construction (length mismatch), why choose GRPO, tool-response loss masking, MCP construction count/cases, and “most memorable bad case.” [61]
Type 1: A 90-minute interview post includes “tokenizer + RAG pseudo-code,” KL estimation questions, and “DeepSeek-R1 training flow + MLA.” [62]

StepFun
Type 1: A recent post drills GAE, MC vs TD, bias/variance, “GRPO advantage and baseline,” and RL+MoE issues. [63]

Xiaohongshu (via public reprint)
Type 1: A multimodal interview lead emphasizes training data quality, LLM-based data synthesis, Self-Instruct vs RLAIF, RLHF stability, GPU utilization issues, and RM training difficulty. [9]

SenseTime
Type 1: A 2026-03 internship interview list asks memory optimization, attention math intuition, PT/SFT/RL relationships, batch-size & LR interplay, and doc dedup, with on-the-spot coding. [64]

iFLYTEK
Type 1: A 2026-03 interview explicitly tests PPO vs GRPO (including clip meaning), DeepSpeed ZeRO-3 improvements, and how you combine verl with vLLM. [65]

Xpeng
Type 1: A multimodal algorithm interview asks vLLM/PagedAttention basics, RL algorithm differences, speculative decoding / reject sampling, and domestic GPU optimization. [66]

Theme-organized question bank

Below are representative questions per theme. Wording is **lightly normalized** to avoid cross-post duplication and to keep the focus on the underlying interviewer intent; keywords (GRPO/PPO/DPO/verl/MCP/...) are preserved.

Post-training / RL fundamentals 1. Explain the full post-training pipeline: SFT → preference training → RL (what changes at each stage, and why). [67]
2. Why is RLHF still used after SFT—when does SFT-only fail? [68]
3. What are GAE, Monte Carlo return, and TD learning; how do bias and variance trade off? [69]
4. Online vs offline RL in your project: what is "the environment," what data do you collect, and what feedback loop exists? [70]
5. Why are "reasoning" abilities becoming a focus, and how can post-training push reasoning? [71]

RL algorithm details (PPO / DPO / GRPO / DAPO / GSPO / DSPO…) 1. Compare PPO vs DPO vs GRPO: what is optimized, what data is required, and what breaks in practice? [72]
2. Explain why PPO uses clipping; what failure does clip mitigate? [73]
3. GRPO: how is the group-relative advantage computed; what is the baseline; why does group sampling matter? [74]
4. GSPO vs GRPO: what changes conceptually (token-level vs sequence-level alignment), and what stability/efficiency tradeoffs are claimed? [75]
5. DPO: what does the loss "mean" intuitively; what happens when preferences are noisy or lengths differ? [76]
6. Practical: why can DPO lead to output length growth, and what mitigations exist? [77]
7. In real work, how do you decide between "train a reward model + RL" vs "direct preference optimization"? [78]

Reward / advantage / critic / KL / entropy / reward hacking 1. How did you design the reward function and why; what is the failure mode if you change weights? [79]
2. Why might you still need a critic/value model if you have a reward model? [80]
3. Where does the KL constraint come from (reward-side vs loss-side); what happens if KL is too small/too large? [81]
4. What does "reward collapse / reward function collapse" look like in logs; how do you debug? [82]
5. How do you detect and prevent reward hacking (or "model gaming the metric") in post-training? [83]

verl / vLLM / DeepSpeed / Ray / distributed training & training systems 1. Walk through verl's GRPO training flow end-to-end (rollout → scoring → advantage → update). [84]
2. Explain how rollout.n and train batch size determine the number of trajectories; relate it to update mini-batches and epochs. [85]
3. What is remove-padding and why does it matter for efficiency? [86]
4. DeepSpeed ZeRO stages 1–3: what state is partitioned; how does it change memory and comm? [87]
5. MoE train–serve mismatch: what forms does mismatch take, why is it worse for RL, and how would you mitigate? [88]
6. How do you integrate verl training with vLLM serving for rollouts; what is the division of labor? [89]
7. If GPU utilization is low or fragmentation is severe, how do you troubleshoot? [90]
8. Ray + DeepSpeed: what does Ray handle (setup/scaling/checkpoint) vs what DeepSpeed handles (optimizer/memory)? [91]

Agent / tool use / planning / memory / evaluation / safety 1. Explain a modular agent's multi-step planning loop; how do you represent plans and revisions? [92]

2. Tool-chain scheduling: how do you choose tools, order them, and design fallback when tools fail? [93]
3. Agent evaluation: what metrics separate planning quality from hallucination rate, and what's your gate? [94]
4. Memory update/forgetting: when do you write, rerank, summarize, or discard memory? [95]
5. Tool-trace training: do you mask loss on tool responses; why? [96]
6. MCP: what problem does it solve vs function calling; what's the basic client–server flow? [97]
7. If an agent pipeline has multiple tools and high QPS but latency is high, what do you optimize first (model, retrieval, tool, caching, concurrency)? [98]

RAG / retrieval / rerank / long context 1. Describe a standard RAG pipeline and how you validate it "works" (offline/online). [99]
2. Embedding-first retrieval vs rerank: why choose rerank; what is the cost/benefit? [100]
3. Repeated user questions: how do you reduce wasted retrieval cost (cache, canonicalization, routing)? [101]
4. How do you handle recall errors (漏召) and wrong recalls (误召); what knobs do you tune? (More common in compiled agent interviews.) [102]
5. Long context vs RAG: when does long context win; what evaluation tells you? [103]

Transformer / MoE / attention / RoPE / RMSNorm, etc. 1. Explain attention "mathematically" and intuitively; what does softmax do; why dot-product? [104]
2. RMSNorm vs LayerNorm: difference and why modern LLMs prefer one. [105]
3. Pre-Norm vs Post-Norm: how does it affect stability? [106]
4. GQA / MQA / MLA: what is shared, why does it reduce KV cache, and what tradeoffs appear? [107]
5. MoE: dense vs MoE tradeoff; load balancing; why MoE is preferred for scaling. [108]
6. FlashAttention: what bottleneck does it address? [109]

Inference & serving / latency / cost / KV cache / PagedAttention 1. KV cache: what is stored, how it grows, and how it determines memory and throughput. [110]
2. PagedAttention: why paging helps reduce fragmentation and improves throughput. [111]
3. Continuous batching and prefix caching: when they help, when they hurt. [112]
4. Speculative decoding / reject sampling: what is the formula-level idea and what limits it? [113]
5. GPU memory management design for serving: what structures and policies would you implement? [114]

Project deep dive 1. What was the real business objective and how did you measure success (online traffic, DAU, cost)? [115]
2. How did you build data (SFT/RL) and validate data quality; how did you avoid bias amplification or self-reinforcement? [116]
3. How did you auto-judge multiple candidates (responses) and ensure judge alignment with humans? [117]
4. What were the worst "bad cases," what did you change, and what metric moved? [118]
5. Why did you pick Qwen vs internal models; what was the ablation? [119]

Live coding / DS&A Examples observed include: MHA implementation, 2D prefix sums, islands counting, longest common substring, "K-group reverse linked list," etc. [120]

Final top question lists

These lists are **weighted toward Type 1 real interview experiences** (explicit dates/rounds) and de-duplicated by intent. Items marked "(weaker authenticity)" are mainly seen in compilations/templates rather than multiple real rounds.

Post-training and agentic RL high-frequency Top 50

1. PPO vs DPO vs GRPO: compare objectives, data needs, and practical pitfalls.
2. PPO clipping: why the surrogate objective uses clip; what instability it prevents.
3. GRPO: how group sampling is set up; what "baseline" is; how advantage is computed.
4. GSPO vs GRPO: what changes (token-level vs sequence-level); why stability might improve.
5. DAPO/GSPO: what is the claimed "improvement direction," and what new failure can appear.
6. In your project: why you chose GRPO (or DPO) instead of alternatives; what was compared.
7. DPO loss intuition: what gradients are doing when preferred vs rejected log-probs change.
8. Why DPO can lead to longer outputs; how to control length without killing quality.
9. Offline vs online RL: in LLM post-training, what does "environment interaction" mean?
10. RL data volume vs SFT data volume: how do you choose; how do you explain scaling.
11. How to design reward functions: what signals; how to weight; how to avoid shortcut learning.
12. Why critic/value models exist; when RM alone is not enough.
13. Reward hacking: how it happens even without an explicit reward model; how to detect.
14. "Reward collapse" / "reward function collapse": what it looks like, why it happens, how to debug.
15. If GRPO training "loss becomes ~0": plausible reasons (data, mask, KL, scaling) and fixes.
16. What is KL control doing (reward-side penalty vs loss-side KL term)?
17. What happens if KL is too small vs too large; what's your monitoring strategy.
18. How do you evaluate alignment improvements offline (benchmarks, judge models, humans)?
19. How do you ensure the judge/auto-eval correlates with human preference?
20. How do you build negative samples that are "plausible but wrong" (hard negatives)?
21. For reasoning tasks: how to handle CoT length exploding; do you compress, truncate, or re-train?
22. What is GAE; why it reduces variance; how it connects to advantage estimation.
23. Monte Carlo vs TD: bias/variance trade-offs; where you would use each.
24. Why "reasoning capability" became a focus; what training signals support reasoning.
25. In R1-style training: what is "verifiable reward" style (correctness-only reward), and trade-offs.
26. How do you set batch size vs learning rate; which you tune first and why.
27. How do you diagnose SFT loss oscillation (data, LR schedule, padding mask, batch composition)?
28. How do you do padding loss masking (esp. in SFT and tool traces)?
29. Multi-objective rewards: how do you resolve reward conflicts; what if objectives fight?
30. What is entropy regularization; what is "entropy collapse," and when does it matter? (weaker authenticity)
31. How do you monitor policy drift and prevent model quality regressions?
32. If model starts repeating ("repetition / mode collapse"), what levers do you pull? (partly weaker authenticity)
33. Data filtering for post-training: which dimensions do you filter on; how to validate alignment.
34. Can model-based filtering match human filtering; how to estimate mismatch.
35. "Question difficulty": how do you define it; how do you build a router/classifier to route policies.
36. Two-stage RL / multi-phase RL: what's different across stages; why it might help.
37. How to connect RL training to production constraints (latency, cost, throughput).
38. "How many GPUs/nodes did you use": justify resources and efficiency choices.
39. DeepSpeed ZeRO: what stage you used and why; what the memory/comm breakdown is.
40. MoE + RL: what new stability/consistency problems appear?

41. MoE train–serve mismatch: how do you detect it; what mitigation exists?
42. What is vLLM and why it matters for rollouts/serving; what does PagedAttention solve?
43. KV cache sizing: estimate memory cost given layers/hidden dims/seq length.
44. Speculative decoding / reject sampling: how it speeds up; what requirements it has.
45. “Tool response loss mask”: do you train on tool outputs; why/why not?
46. MCP in training data: how do you construct tool-calling traces; what formats and edge cases?
47. “Agentic training average output tokens”: how measured; whether you compress single responses.
48. A/B testing: how you design it, what metrics, and how you ensure causality in improvements.
49. Judge model / evaluation model design: how you build it and avoid bias.
50. “Most memorable bad case”: what it revealed and what you changed afterwards.

Evidence base: real-round question lists and drill-down patterns appear repeatedly in Ant, Tencent, ByteDance, Meituan, Zhipu, StepFun, iFLYTEK, Amap, and others; plus primary algorithm sources for PPO/DPO/GRPO. [121]

Must-master project drill-down Top 20

1. State the business objective (what problem) and the measurable success metric (offline + online). [122]
2. What is the end-to-end pipeline (data → training → evaluation → deployment), with exact boundaries. [123]
3. What data sources did you use; how did you clean, deduplicate, and filter them? [124]
4. How did you build preference pairs or rankings; what is the negative-sample strategy? [125]
5. How did you generate synthetic data; how did you prevent self-reinforcing bias? [126]
6. How do you auto-judge multi-sample candidates; how do you validate judge reliability vs humans? [117]
7. What was your reward function; why those terms; what ablations support it? [79]
8. What RL algorithm did you choose and why; what were the baselines? [127]
9. What training resources did you use (GPU count, nodes, batch sizes) and why those settings? [128]
10. What failure did you hit first (NaN, loss=0, reward drop); how did you debug step-by-step? [129]
11. If output got too long, too repetitive, or too “safe,” what knob did you adjust and why? [130]
12. What is your offline evaluation suite (benchmarks, long context, safety), and what is the “ship gate”? [131]
13. How do you ensure improvements are not reward hacking; what counter-evals exist? [132]
14. What are your best/worst cases; how do they map back to data or reward issues? [96]
15. If you used RAG + agent: what is your retrieval cost and caching strategy for repeated queries? [101]
16. How do you measure latency/cost end-to-end; what is the biggest bottleneck? [98]
17. Why choose an external base model (e.g., Qwen) vs an internal one; what constraints drove it? [119]
18. How many iterations did you run; what changed between versions; what was each version’s lesson? [61]
19. How do you do A/B testing (allocation, metrics, duration, confounders)? [133]
20. What is the most “engineering” part you owned (serving, infra, data ops) and why it mattered? [134]

verl / RL infra / distributed training high-frequency Top 20

1. Explain verl GRPO training loop; where are actor/reference, how is KL applied. [135]
2. Explain rollout.n, train batch size, and how many trajectories you generate per update. [136]
3. Explain how ppo_mini_batch_size relates to global batch and gradient accumulation. [137]
4. Explain ppo_micro_batch_size_per_gpu and why it is about memory not convergence. [138]

5. How do rollout batch sizes translate into “samples per GPU per update” (hands-on arithmetic). 51
6. When do you use KL-in-loss vs KL-in-reward; how do you tune coefficient. 139
7. Loss aggregation modes (token-mean vs seq-mean-token-mean): why long-CoT stability differs. 140
8. remove-padding: what it does and how to validate correctness. 86
9. DeepSpeed ZeRO 1/2/3: partition targets and memory/comm tradeoffs. 141
10. DeepSpeed ZeRO-3 “what improved” and what new bottlenecks appear. 142
11. FSDP vs ZeRO: how they differ operationally (wrapping, sharding, offload). 143
12. Tensor/sequence parallelism: what is communicated and why; where deadlocks can happen. 144
13. MoE RL training: what instability/mismatch issues you see and instrumentation needed. 144
14. “GPU utilization low / memory fragmentation high”: debug checklist and likely culprits. 90
15. How does vLLM integrate with RL rollouts; which part owns sampling and caching. 145
16. KV cache size estimation for a given model; what knobs reduce it (GQA/MQA/MLA). 146
17. PagedAttention: what fragmentation it solves; how paging maps to GPU blocks. 111
18. Continuous batching and scheduling policies; prefill vs decode interplay. 112
19. Fault tolerance: what happens if a node fails mid-training; checkpoint strategy. 147
20. Ray + DeepSpeed: what Ray automates; how to debug multi-node environment issues. 91

Agent / tool use / coding agent high-frequency Top 20

1. Agent vs function calling: what is the minimal closed loop and what is “agentic” control. 148
2. Tool selection: how you describe tools; how tool schema quality affects behavior. 149
3. Tool-chain scheduling: how you route among tools; how you design fallback and retries. 93
4. Multi-step planning: how a modular agent plans, revises, and stops. 92
5. Memory design: write/update/forget cycles; summarization vs retrieval memory. 95
6. Agent evaluation: dimensions and metrics; separating planning from hallucination. 131
7. Latency optimization when multiple tools + high QPS; where caching helps most. 150
8. MCP: what it standardizes; how it differs from per-tool integration. 151
9. MCP in projects: what MCP tools you built and how you tested them. 152
10. MCP vs “Skills”: what’s reusable prompt/process vs tool protocol. (mostly template-driven, but asked) 153
11. ReAct: what are the modes and why interleaving action reduces hallucination. 154
12. Tool-trace supervision: do you mask loss on tool responses; why it matters. 96
13. Training “tool calling” data: how you construct traces and edge cases. 155
14. Long tool chains: how to avoid context blow-up; summarization and compression. 156
15. Coding agent evaluation: how you verify code correctness (tests, unit-test rewards). (weaker authenticity; appears mostly in agent-dev posts) 157
16. SWE evaluation: what benchmarks like SWE-bench measure; what “Verified” means. 158
17. Using IDE agents (e.g., Cursor) effectively: how you deal with repeated incorrect edits. (app-dev oriented) 159
18. Designing a “fully automated” agent workflow: what are the top three failure points. 102
19. Tool failure handling: timeouts, partial results, recovery, and user trust impact. 93
20. Security in tool connections: prompt injection and tool abuse risks. (weaker authenticity in interview logs, but rising in MCP discourse) 160

General LLM fundamentals supplement Top 30

1. Attention: derive formula; explain what softmax does and why dot-product. 104
2. Multi-head attention vs single-head; what changes in representation and compute. 161
3. Decoder-only vs encoder-decoder: when each is used and why decoder-only dominates LLMs. 162

4. RoPE: core idea and why it supports extrapolation. [21]
5. RMSNorm vs LayerNorm differences; why RMSNorm is popular. [163]
6. Pre-Norm vs Post-Norm; effects on stability. [106]
7. FlashAttention: what memory bottleneck it solves. [109]
8. Mamba: what problem it tries to solve vs attention-based transformers. [109]
9. MoE: dense vs MoE tradeoff and scaling rationale. [164]
10. MoE load balancing and auxiliary losses; what happens if experts collapse. [164]
11. GQA/MQA/MLA: relationship to KV cache reduction. [165]
12. KV cache: what it stores; how it scales with seq length and batch. [166]
13. PagedAttention: paging analogy; why it reduces fragmentation. [111]
14. Continuous batching: how it improves throughput; fairness issues. [112]
15. Prefix caching: when it works (shared prefixes) and invalidation issues. [112]
16. Speculative decoding: acceptance/reject mechanics. [113]
17. Quantization basics (INT8/FP16/BF16/FP8) and why chosen. [167]
18. LoRA: why low-rank works; how you choose rank/alpha. [168]
19. QLoRA / NF4 idea. (more compilation-driven) [21]
20. Gradient explosion/vanishing: diagnosis and fixes. [62]
21. Batch size vs LR: coupled tuning intuition. [169]
22. How to estimate parameter count from layer count and hidden size. [170]
23. Tokenizer basics: what output is; what BPE vs unigram means. [171]
24. What is “hallucination” and how to mitigate (RAG, calibration, training). [172]
25. Why lowering temperature can reduce hallucination; what tradeoff it introduces. [173]
26. PT vs SFT vs RL relationships; when stages can/can’t substitute. [174]
27. Document dedup (minhash, exact, semantic) and why it matters. [175]
28. Long context: what breaks (attention cost, KV cache), what mitigates (chunking, RAG). [176]
29. Loss masking (padding, tool traces) and why incorrect mask ruins training. [177]
30. Compute/memory math for serving and training; show your “back-of-the-envelope” reasoning. [178]

One-week and two-week prep roadmap

This roadmap is optimized for your stated preference: **post-training → GRPO variants → verl infra → tool-use agents → code-capability training & verifiable rewards**.

If you have one week (7 days): focus on S-level only - **Days 1–2: GRPO/PPO/DPO core mechanics + failure modes.**
Learn PPO clipping and KL constraint from primary sources; learn DPO’s “no explicit RM” formulation; learn GRPO from DeepSeekMath and how modern reasoning RL uses correctness-based rewards. Then immediately map these to interview-shaped prompts: “PPO vs GRPO,” “why DPO makes outputs longer,” “advantage baseline,” “reward collapse.” [179]
- **Days 3–4: verl configs + DeepSpeed ZeRO + memory math.**
Treat verl docs as a “whiteboard config spec.” You should be able to explain rollout.n, train_batch_size, ppo_mini_batch_size, ppo_micro_batch_size_per_gpu, loss_agg_mode, and use_kl_loss without looking. Add ZeRO stage reasoning (what is sharded, what communication happens) from the ZeRO paper and DeepSpeed docs usage patterns. [180]
- **Days 5–6: tool-use agent loop + MCP + evaluation gates.**
Use ReAct to anchor “reason+act trajectories,” then learn MCP’s purpose and request-response flow, and practice answering: scheduling, fallback, memory update, evaluation metrics (“planning vs hallucination”), and how to test tool-calling. Tie back to real interview prompts (Taobao agent, Zhipu agent). [181]

- **Day 7: drill your own project story as if an interviewer is hostile.**
Prepare a structured narrative: objective → data → method choice → ablations → offline eval → online metrics → failure cases → fixes. Make sure you can answer "how many GPUs," "how you built negatives," "how you judge candidates," and "what bad case taught you." [182]

If you have two weeks (14 days): add A-level breadth and conversion to "interview proof" - Week 2, Days 8–10: serving/inference essentials for post-training engineers.
Learn vLLM/PagedAttention's memory model, KV cache sizing, and continuous batching/prefix caching (because these appear both in real infra interviews and in curated frequency summaries). Practice explaining latency/cost tradeoffs with numbers. [183]
- **Week 2, Days 11–12: MoE + train–serve mismatch + RL stability.**
Prepare answers for "MoE under RL," load balancing, and mismatch cases; connect to observed interview prompts that explicitly ask "MoE train–serve inconsistency" and "reward suddenly drops." [184]
- **Week 2, Days 13–14: evaluation and coding-agent realism.**
Prepare a compact story for coding-agent training and evaluation using public benchmarks (e.g., SWE-bench) as "credible evidence," and be ready for "how do you verify AI-generated code" style questions that appear in agent-dev interviews. [185]

---

[1] [2] [5] [12] [16] [18] [24] [26] [27] [50] [51] [72] [78] [79] [80] [84] [85] [121] [127] [137] 蚂蚁金服一二面面经_牛客网
https://www.nowcoder.com/feed/main/detail/c1f9b7cec4eb4d1ea27f86416daa89f0

[3] [29] [47] [67] [92] [93] [94] [98] [99] [131] [149] [150] 大模型Agent校招面经-阿里淘天_牛客网
https://www.nowcoder.com/feed/main/detail/336ecac3af1e48dea069913b1dc26e83?sourceSSR=%E5%85%B6%E4%BB%96

[4] [37] [39] [42] [75] [115] [116] [117] [120] [128] [129] [133] [182] WXG 大模型算法 实习面经_牛客网
https://www.nowcoder.com/feed/main/detail/84f86b0fa61d4563b6ac6f3bc74c0d67

[6] 通过手搓一个Agent, 我的AI精神内耗治好了
https://juejin.cn/post/7613709178717945891?utm_source=chatgpt.com

[7] [28] [54] https://github.com/verl-project/verl
https://github.com/verl-project/verl

[8] 一文搞懂大模型强化学习策略：DPO、PPO和GRPO
https://zhuanlan.zhihu.com/p/29033948405?utm_source=chatgpt.com

[9] [90] [108] [124] [126] [164] 算法实习面经-小红书多模态一面_牛客网
https://www.nowcoder.com/feed/main/detail/bc194d18c3dc4842978980abb7e6a108

[10] [15] [25] [40] 腾讯大模型实习一面 1h_牛客网
https://www.nowcoder.com/feed/main/detail/046ac6fcaf4c4b0696c6573416712416?sourceSSR=enterprise

[11] [45] [86] [87] [88] [144] [184] 文心基模 AI infra面经 实习面经_牛客网
https://www.nowcoder.com/feed/main/detail/6a91f255d42d47b6a30d828adc7265ba?sourceSSR=%E5%85%B6%E4%BB%96

[13] [44] [71] [77] [83] [130] [132] 百度-文心一言-一面面经_牛客网
https://www.nowcoder.com/discuss/730781226709061632

[14] [实习面经] minimax 大模型算法_牛客网
https://www.nowcoder.com/feed/main/detail/c756dc7f89054486ae54065e02556a61

[17] [61] [76] [96] [97] [118] [125] [146] [155] [170] 智谱大模型agent算法面经 好细啊_牛客网
https://www.nowcoder.com/feed/main/detail/24952c828c59435abd3c302c97fa358d

19 35 112 114 134 178 2025春招实习面经汇总（已接阿里offer）_牛客网
https://www.nowcoder.com/discuss/736868736837173248

20 21 49 103 106 https://www.nowcoder.com/discuss/848942791164981248
https://www.nowcoder.com/discuss/848942791164981248

22 63 69 74 167 阶跃星辰大模型算法实习面经 好难_牛客网
https://www.nowcoder.com/feed/main/detail/933afb27edd84b438682fd5bba95fbb3?sourceSSR=dynamic

23 https://arxiv.org/abs/2501.12948
https://arxiv.org/abs/2501.12948

30 https://modelcontextprotocol.io/specification/2025-06-18
https://modelcontextprotocol.io/specification/2025-06-18

31 46 95 100 101 119 123 淘天 Agent 面经_牛客网
https://www.nowcoder.com/feed/main/detail/485fbcf14893475a8dbb137064ea34f5

32 181 https://arxiv.org/abs/2210.03629
https://arxiv.org/abs/2210.03629

33 34 55 105 109 163 美团大模型强化学习一面-实习面经_牛客网
https://www.nowcoder.com/feed/main/detail/46561023362b426db761cbee0fea4611

36 166 176 https://github.com/vllm-project/vllm
https://github.com/vllm-project/vllm

38 我是如何用AI 写了70% 的逻辑却被面试官夸“基本功扎实”的 ...
https://www.nowcoder.com/feed/main/detail/a97b1cbb5e6a40519291f3313b971fde?toCommentId=22489815&utm_source=chatgpt.com

41 156 腾讯大模型算法校招凉经_牛客网
https://www.nowcoder.com/feed/main/detail/3358b976c5e946a3989fbb61e0bc11e2?urlSource=home-api

43 68 腾讯-混元大模型面经-华5硕_牛客网
https://www.nowcoder.com/feed/main/detail/181b9457619941fdbcf32b3fbf067dcd

48 177 https://www.nowcoder.com/feed/main/detail/23044dfa6a5d4274a84632ce50941692
https://www.nowcoder.com/feed/main/detail/23044dfa6a5d4274a84632ce50941692

52 70 82 161 字节大模型实习算法岗面经_牛客网
https://www.nowcoder.com/feed/main/detail/bda49826f8cc41c992ad630a3e8255de

53 重在参与:字节多模态大模型算法岗面经_牛客网
https://www.nowcoder.com/feed/main/detail/7604918efc574981a47d32337939a6a3

56 美团算法大模型实习二面 70min_牛客网
https://www.nowcoder.com/feed/main/detail/7b57311582514446b84a28dee00bf09e?sourceSSR=dynamic

57 107 110 165 快手日常实习大模型算法面经_牛客网
https://www.nowcoder.com/feed/main/detail/9d24592752804d2790557f81230df62d

58 大模型面经-快手_牛客网
https://www.nowcoder.com/feed/main/detail/e3fb84fa4cdd44878199ebeff2f3a1fb

59 快手二面
https://www.nowcoder.com/feed/main/detail/c7d3992e36b44234917382c3b7573a00?utm_source=chatgpt.com

60 122 minimax大模型算法实习二面 攒人品_牛客网
https://www.nowcoder.com/feed/main/detail/a28b6cbc9ec04a8a8fd4cb3d26fd6537

62 171 90分钟面经:智谱大模型_牛客网
https://www.nowcoder.com/feed/main/detail/e546ae199c7c417d88c9b468e42a6d50

64 104 169 174 175 商汤算法大模型一面-实习面经_牛客网
https://www.nowcoder.com/feed/main/detail/abdc619dab3f45ad8c6fa9c3acfddb16

65 73 89 142 145 168 172 173 日常实习 科大讯飞 大模型算法一面_牛客网
https://www.nowcoder.com/feed/main/detail/feedea2675d442fe8e553829e5b29e61?sourceSSR=%E5%85%B6%E4%BB%96

66 113 小鹏汽车多模态大模型算法面经_牛客网
https://www.nowcoder.com/feed/main/detail/96f931642588471e8939e29028604499

81 135 136 139 140 180 https://verl.readthedocs.io/en/latest/algo/grpo.html
https://verl.readthedocs.io/en/latest/algo/grpo.html

91 147 https://docs.ray.io/en/latest/train/deepspeed.html
https://docs.ray.io/en/latest/train/deepspeed.html

102 148 152 154 157 159 AI agent应用开发一面面经_牛客网
https://www.nowcoder.com/feed/main/detail/416c3118a9c84fe0a43dc00f9eb9bec2

111 183 https://arxiv.org/abs/2309.06180
https://arxiv.org/abs/2309.06180

138 143 https://verl.readthedocs.io/en/latest/examples/config.html
https://verl.readthedocs.io/en/latest/examples/config.html

141 https://arxiv.org/abs/1910.02054
https://arxiv.org/abs/1910.02054

151 https://www.anthropic.com/news/model-context-protocol
https://www.anthropic.com/news/model-context-protocol

153 面试官视角聊聊：秋招AI岗高频面试问题
https://www.nowcoder.com/discuss/857300424817160192?utm_source=chatgpt.com

158 185 https://www.swebench.com/SWE-bench/
https://www.swebench.com/SWE-bench/

160 https://www.anthropic.com/engineering/code-execution-with-mcp
https://www.anthropic.com/engineering/code-execution-with-mcp

162 字节-搜推大模型-实习面经-一面记录（强度太高了！）_牛客网
https://www.nowcoder.com/feed/main/detail/e138d6faa3eb4234960a5c4d912a8efa

179 https://arxiv.org/abs/1707.06347
https://arxiv.org/abs/1707.06347